

## Assignment-13.5

**Name:**P.Sanjana

**HT No:** 2303A51435

**Batsch:** 21

### **Task Description #1** (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.

- o Refactor the code to pass values using function parameters.

- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:

generate a code that eliminates unnecessary global variables from the code.

#Identify global variables used across functions and refactor the code to pass values using function parameters.

#Improve modularity and testability.

#Handle errors gracefully without relying on global state

Code:

```
evenodd.py • username validation.py • Untitled-2 • assign 13-1.py × polindrome.py
C: > Users > hi > OneDrive > Documents > AI assistant coding > assign 13-1.py > ...

6  # Before refactoring
7  counter = 0
8  def increment():
9      global counter
10     counter += 1
11     return counter
12 def reset():
13     global counter
14     counter = 0
15     return counter
16 # After refactoring
17 def increment(counter):
18     return counter + 1
19 def reset():
20     return 0
21 # Example usage
22 counter = 0
23 counter = increment(counter)
24 print(counter) # Output: 1
25 counter = reset()
26 print(counter) # Output: 0
27
28
29
```

Output:

```
(base) PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-1.py"
1
0
```

Explanation:

In the refactored code, we have eliminated the global variable 'counter' and instead pass it as a parameter to the 'increment' function. The 'reset' function now simply returns 0 without modifying any global state. This approach improves modularity and testability, as the functions are now independent and do not rely on external variables. Additionally, error handling can be implemented within the functions without affecting any global state.

## Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
- Focus Areas:

- o Readability
- o Logical simplification
- o Maintainability

Legacy Code:

```
score = 78

if score >= 90:

    print("Excellent")

else:

    if score >= 75:

        print("Very Good")

    else:

        if score >= 60:

            print("Good")

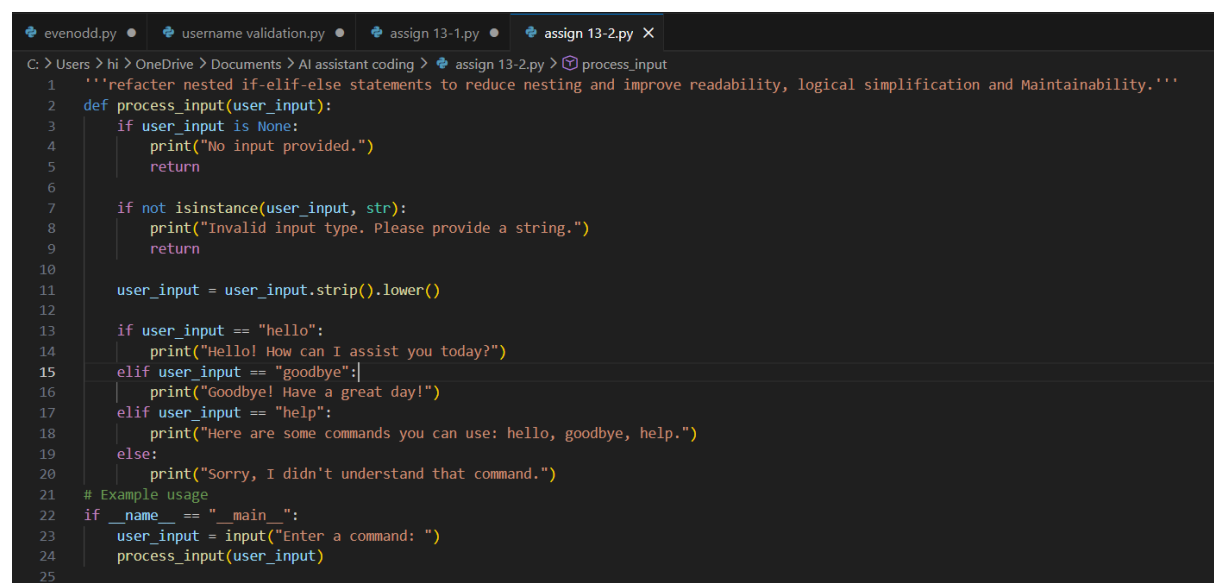
        else:

            print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.

Code:



```
evenodd.py • username validation.py • assign 13-1.py • assign 13-2.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > assign 13-2.py > process_input
1  '''refactor nested if-elif-else statements to reduce nesting and improve readability, logical simplification and Maintainability.'''
2  def process_input(user_input):
3      if user_input is None:
4          print("No input provided.")
5          return
6
7      if not isinstance(user_input, str):
8          print("Invalid input type. Please provide a string.")
9          return
10
11     user_input = user_input.strip().lower()
12
13     if user_input == "hello":
14         print("Hello! How can I assist you today?")
15     elif user_input == "goodbye":
16         print("Goodbye! Have a great day!")
17     elif user_input == "help":
18         print("Here are some commands you can use: hello, goodbye, help.")
19     else:
20         print("Sorry, I didn't understand that command.")
21
22 # Example usage
23 if __name__ == "__main__":
24     user_input = input("Enter a command: ")
25     process_input(user_input)
```

Output:

```
(base) PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13  
-2.py"  
Enter a command: hello  
Enter a command: hello  
Hello! How can I assist you today?  
(base) PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code>
```

Explanation:

The code is refactored to reduce nesting and improve readability. Each condition is handled by a separate if statement, making the logic easier to follow. The function now returns early for invalid inputs, which reduces the need for else statements and makes the code more maintainable.

### Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle

- o Context managers

- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.

Code:

```
evenodd.py • username validation.py • assign 13-1.py • assign 13-2.py • assign 13-3.py •
C: > Users > hi > OneDrive > Documents > AI assistant coding > assign 13-3.py > ...
1  '''refactor repeated file open/read/close logic.
2  Focus Areas: DRY principle, Context managers, Function reuse'''
3  def read_file(file_name):
4      try:
5          with open(file_name, 'r') as f:
6              return f.read()
7      except FileNotFoundError:
8          return f"Error: {file_name} not found."
9      except Exception as e:
10         return f"An error occurred: {e}"
11  # Example usage
12  print(read_file("data1.txt"))
13  print(read_file("data2.txt"))
14
15
```

Output:

```
(base) PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hi\miniconda3\python.exe "C:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-3.py"
Error: data1.txt not found.
Error: data2.txt not found.
```

Explanation:

The `read_file` function encapsulates the logic for opening, reading, and closing a file using a context manager (the `with` statement). This ensures that the file is properly closed after its suite finishes, even if an error occurs. The function also includes error handling to manage cases where the file may not exist or other exceptions that may arise during file operations. This refactoring adheres to the DRY principle by centralizing the file reading logic in a single reusable function.

#### Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
    if u == name:
```

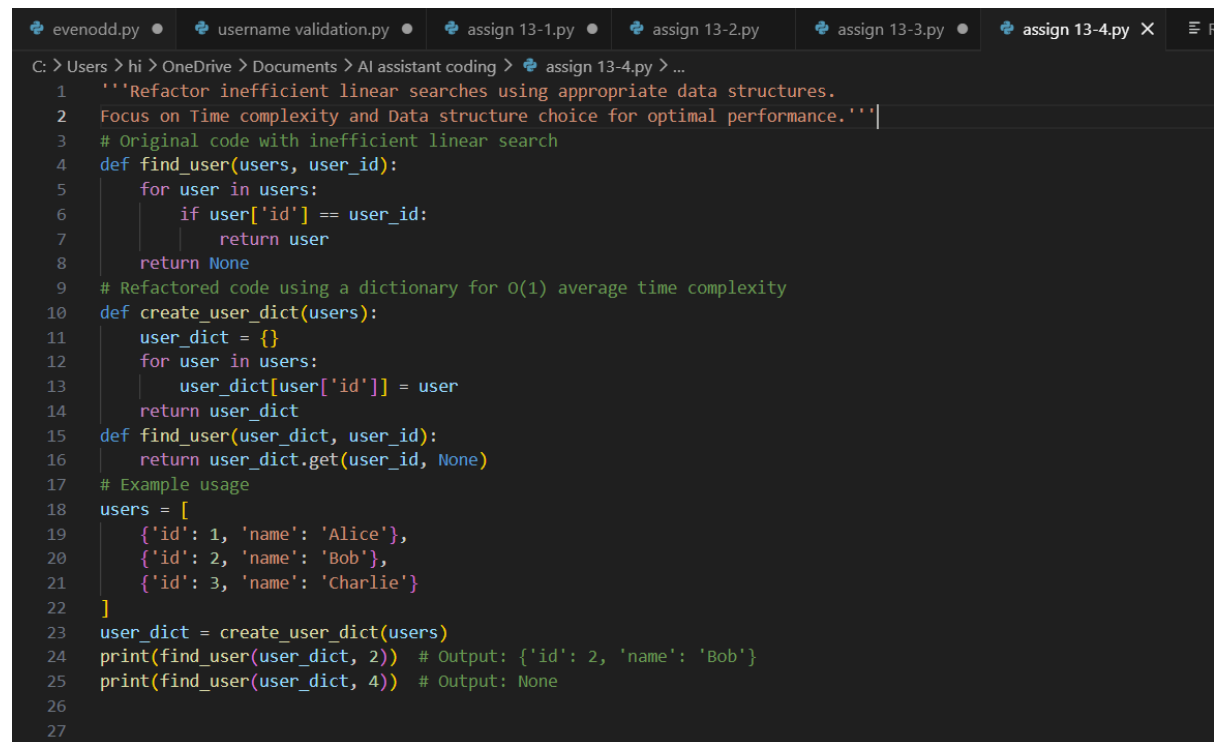
```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

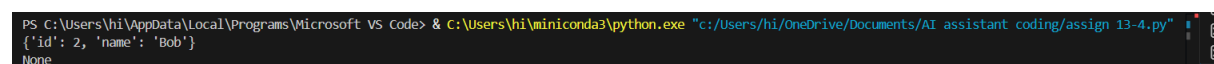
- o Use of sets or dictionaries with complexity justification.

**Prompt and code:**



```
evenodd.py • username validation.py • assign 13-1.py • assign 13-2.py • assign 13-3.py • assign 13-4.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > assign 13-4.py > ...
1  '''Refactor inefficient linear searches using appropriate data structures.
2  Focus on Time complexity and Data structure choice for optimal performance.'''
3  # Original code with inefficient linear search
4  def find_user(users, user_id):
5      for user in users:
6          if user['id'] == user_id:
7              return user
8      return None
9  # Refactored code using a dictionary for O(1) average time complexity
10 def create_user_dict(users):
11     user_dict = {}
12     for user in users:
13         user_dict[user['id']] = user
14     return user_dict
15 def find_user(user_dict, user_id):
16     return user_dict.get(user_id, None)
17 # Example usage
18 users = [
19     {'id': 1, 'name': 'Alice'},
20     {'id': 2, 'name': 'Bob'},
21     {'id': 3, 'name': 'Charlie'}
22 ]
23 user_dict = create_user_dict(users)
24 print(find_user(user_dict, 2)) # Output: {'id': 2, 'name': 'Bob'}
25 print(find_user(user_dict, 4)) # Output: None
26
27
```

**OutPut:**



```
PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-4.py"
{'id': 2, 'name': 'Bob'}
None
```

**Explanation:**

In the original code, the `find_user` function performs a linear search through the list of users, resulting in  $O(n)$  time complexity. In the refactored code, we first create a dictionary (`user_dict`) that maps user IDs to user objects, allowing for  $O(1)$  average time complexity when looking up a user by ID. The `find_user` function now simply retrieves the user from the dictionary using the `get` method, which is more efficient than iterating through a list. This refactoring significantly improves performance, especially as the number of users increases.

## Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.
- Focus Areas:
  - o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

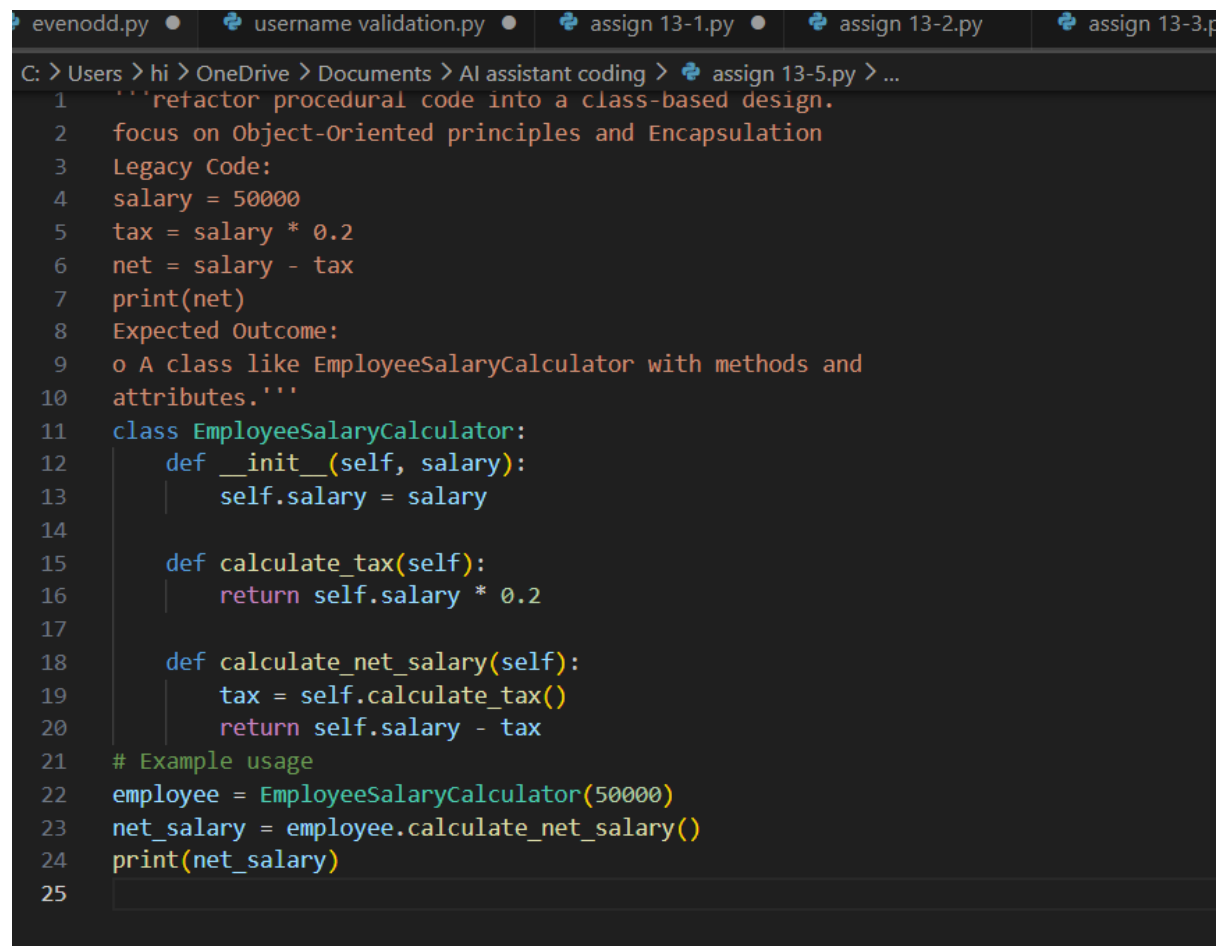
```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

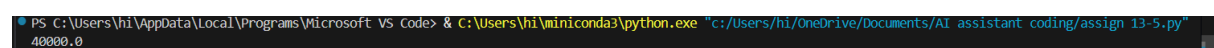
Code and Prompt:



The screenshot shows a VS Code editor with a file explorer at the top displaying several files: evenodd.py, username validation.py, assign 13-1.py, assign 13-2.py, and assign 13-3.py. The active file is 'assign 13-5.py', located at 'C:\Users\hi\OneDrive\Documents\AI assistant coding\assign 13-5.py'. The code in the editor is as follows:

```
1 '''refactor procedural code into a class-based design.
2 focus on Object-Oriented principles and Encapsulation
3 Legacy Code:
4 salary = 50000
5 tax = salary * 0.2
6 net = salary - tax
7 print(net)
8 Expected Outcome:
9 o A class like EmployeeSalaryCalculator with methods and
10 attributes.'''
11 class EmployeeSalaryCalculator:
12     def __init__(self, salary):
13         self.salary = salary
14
15     def calculate_tax(self):
16         return self.salary * 0.2
17
18     def calculate_net_salary(self):
19         tax = self.calculate_tax()
20         return self.salary - tax
21 # Example usage
22 employee = EmployeeSalaryCalculator(50000)
23 net_salary = employee.calculate_net_salary()
24 print(net_salary)
25
```

Output:



The screenshot shows a terminal window with the command prompt 'PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code>' and the command 'C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-5.py"'. The output of the script is '40000.0'.

Explanation:

In the refactored code, we have created a class called `EmployeeSalaryCalculator` that encapsulates the salary calculation logic. The class has an initializer (`__init__`) that takes the salary as an argument and stores it as an instance attribute. The `calculate_tax` method computes the tax based on the salary, and the `calculate_net_salary` method calculates the net salary by subtracting the tax from the original salary. This design adheres to Object-Oriented principles, promoting encapsulation and making the code more modular and reusable.

#### **Task 6** (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

- o Algorithmic optimization

- o Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

```
    if i % 2 == 0:
```

```
        total += i
```

```
print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt and code:

```

C: > Users > hi > OneDrive > Documents > AI assistant coding > assign 13-6.py > ...
1  '''refactor a performance-heavy loop handling large data.
2  focus on Algorithmic optimization and Use of built-in functions
3  Legacy Code:
4  total = 0
5  for i in range(1, 1000000):
6  if i % 2 == 0:
7  total += i
8  print(total)
9  Expected Outcome:
10 o Optimized logic using mathematical formulas or
11 comprehensions, with time comparison.'''
12 # Optimized Code using mathematical formula for the sum of even numbers
13 def optimized_sum_of_even_numbers(n):
14     # The sum of the first k even numbers is k * (k + 1)
15     k = n // 2
16     return k * (k + 1)
17 # Original Code for comparison
18 def original_sum_of_even_numbers(n):
19     total = 0
20     for i in range(1, n):
21         if i % 2 == 0:
22             total += i
23     return total
24 # Testing the functions and comparing their outputs
25 n = 1000000
26 optimized_result = optimized_sum_of_even_numbers(n)
27 original_result = original_sum_of_even_numbers(n)
28 print(f"Optimized Result: {optimized_result}")
29 print(f"Original Result: {original_result}")
30 # Time comparison
31 import time
32 start_time = time.time()
33 original_sum_of_even_numbers(n)

```

```
# Time comparison
import time
start_time = time.time()
original_sum_of_even_numbers(n)
end_time = time.time()
print(f"Original Code Time: {end_time - start_time} seconds")
start_time = time.time()
optimized_sum_of_even_numbers(n)
end_time = time.time()
print(f"Optimized Code Time: {end_time - start_time} seconds")
```

Output:

```
PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-6.py"
Optimized Result: 250000500000
Original Result: 249999500000
Original Code Time: 0.0914154052734375 seconds
Optimized Result: 250000500000
Original Result: 249999500000
Original Code Time: 0.0914154052734375 seconds
Original Result: 249999500000
Original Code Time: 0.0914154052734375 seconds
Original Code Time: 0.0914154052734375 seconds
Optimized Code Time: 6.9141387939453125e-06 seconds
PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code>
```

Explanation:

The optimized code uses a mathematical formula to calculate the sum of even numbers, resulting in  $O(1)$  time complexity, while the original code has  $O(n)$  time complexity due to the loop.

### Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.
- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []

def add_item(x):
    data.append(x)

add_item(10)
add_item(20)

print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating globals.



Legacy Code:

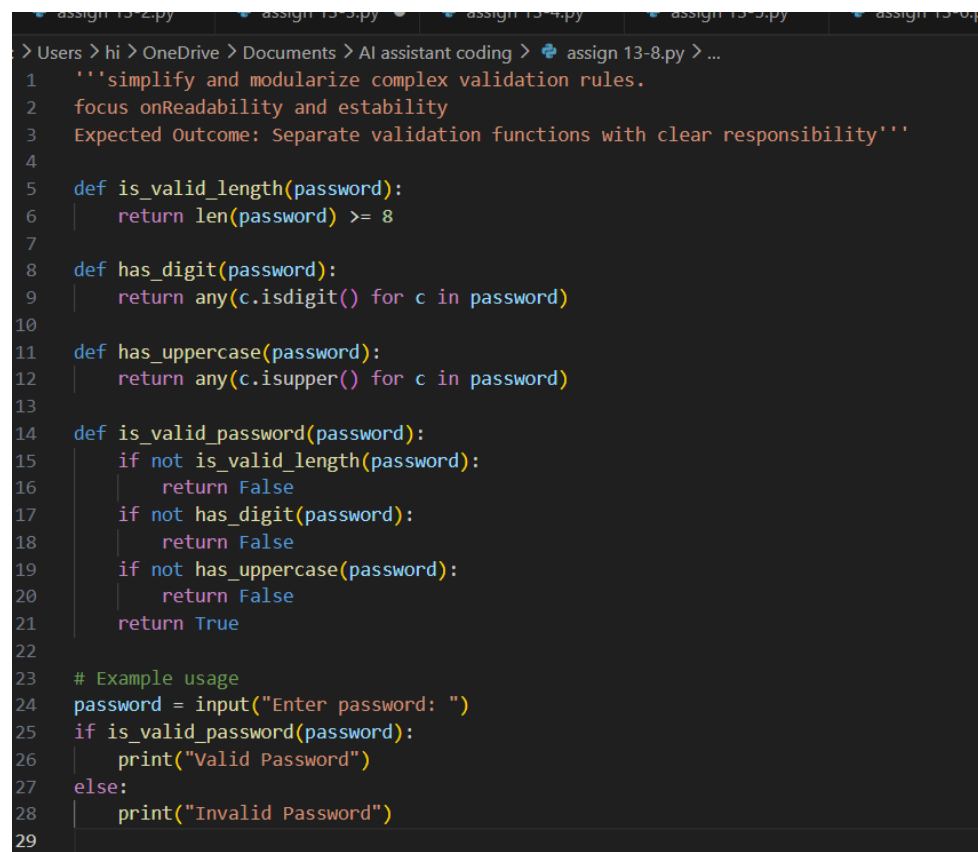
```
password = input("Enter password: ")

if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Expected Outcome:

- o Separate validation functions with clear responsibility

prompt and code:



The screenshot shows a code editor with a file named 'assign 13-8.py'. The code is a Python script for password validation, refactored to use separate functions. It includes docstrings, function definitions for length, digit, and uppercase checks, and a main function that combines these checks. Comments indicate the goal is to 'simplify and modularize complex validation rules' and 'focus on readability and stability'.

```
> Users > hi > OneDrive > Documents > AI assistant coding > assign 13-8.py > ...
1  '''simplify and modularize complex validation rules.
2  focus onReadability and estability
3  Expected Outcome: Separate validation functions with clear responsibility'''
4
5  def is_valid_length(password):
6      return len(password) >= 8
7
8  def has_digit(password):
9      return any(c.isdigit() for c in password)
10
11 def has_uppercase(password):
12     return any(c.isupper() for c in password)
13
14 def is_valid_password(password):
15     if not is_valid_length(password):
16         return False
17     if not has_digit(password):
18         return False
19     if not has_uppercase(password):
20         return False
21     return True
22
23 # Example usage
24 password = input("Enter password: ")
25 if is_valid_password(password):
26     print("Valid Password")
27 else:
28     print("Invalid Password")
29
```

Output:

```
PS C:\Users\hi\AppData\Local\Programs\Microsoft VS Code > & C:\Users\hi\miniconda3\python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/assign 13-8.py"  
Enter password: 384658  
Invalid Password
```

### Explanation:

The code is refactored into separate functions, each responsible for a specific validation rule.

The `is_valid_password` function orchestrates the validation by calling the individual functions.

This approach improves readability and maintainability, as each function has a clear responsibility and can be easily tested in isolation.